EDUASSIST

AI-Powered Student Learning & Study Planner

Academic Project Report

A Generative AI based application for creating student-friendly notes, study plans, summaries and revision support.

Technology: Python • FastAPI • Streamlit • Generative AI

Table of Contents

1. Abstract

2. Introduction

3. Problem Statement

4. Objectives

5. Scope of the Project

6. Existing System

7. Proposed System

8. Key Features

9. User Journey

10. Technology Stack

11. System Architecture

12. Module Description

13. Functional Requirements

14. Non-Functional Requirements

15. Project Workflow

16. AI Prompt Design

17. Backend Development

18. Frontend Development

19. Complete Source Code

20. API Testing

21. Sample Output

22. Error Handling

23. Security and Privacy

24. Testing Strategy

25. Deployment

26. Benefits

27. Limitations

28. Future Enhancements

29. Project Story

30. Conclusion

31. References and Learning Resources

1. Abstract

EduAssist is an AI-powered student learning and study planning application designed to simplify academic preparation. The system accepts basic student information, subject details and a selected topic, then produces structured learning material in a simple and student-friendly format. The application can generate an introduction, important concepts, key points, examples, revision notes and a short summary. A study-planning mode can also organize topics into manageable learning sessions.

The project follows a modular architecture similar to modern AI document-generation systems. A Streamlit interface collects user input, a FastAPI backend validates and processes the request, and a Generative AI component produces the educational content. The generated result is returned to the interface for review and can be prepared for download. The design emphasizes simplicity, modularity, maintainability and responsible use of AI.

2. Introduction

Students often receive large amounts of information from textbooks, lectures and online resources. Finding the important concepts and converting them into revision-friendly material can take considerable time. EduAssist addresses this challenge by providing a single interface for requesting organized learning content.

The system is intended as an educational support tool rather than a replacement for teachers, textbooks or institutional guidance. The student chooses a subject and topic, provides the required level or learning preference, and requests content. The application then creates a structured response that can be read, reviewed and used for revision.

3. Problem Statement

Students may face difficulty when preparing notes for multiple subjects. Common difficulties include identifying key concepts, organizing revision topics, creating short summaries and maintaining a consistent study routine.

The proposed problem is to build a lightweight application that can accept structured student input and automatically create useful study material. The system should provide a clear interface, a simple backend API, AI-assisted generation and readable output.

4. Objectives

The major objectives are:

1. Build a student-friendly learning assistant.
2. Generate structured notes from a subject and topic.
3. Provide concise explanations and examples.
4. Generate a basic study plan based on selected topics.
5. Separate frontend, backend and AI-generation responsibilities.
6. Demonstrate a practical integration of Python, FastAPI, Streamlit and Generative AI.
7. Provide validation and error handling.
8. Keep the application easy to extend with additional learning features.

5. Scope of the Project

EduAssist can be used for topic explanations, revision notes, summaries, key-point extraction and basic study planning. It can be adapted for different subjects and academic levels by changing the prompt and input fields.

The project scope includes a Streamlit user interface, FastAPI API endpoints, request validation, AI prompt construction, response display and optional export functionality. Advanced institutional features such as official grading, examination prediction and automatic certification are outside the current scope.

6. Existing System

In a traditional workflow, a student searches several sources, reads the material, identifies important points, writes notes and separately creates a study timetable. This process can be repetitive.

Existing note-taking applications generally help users store content, while search engines provide access to information but do not necessarily produce a consistent personalized study structure. EduAssist combines structured input and AI-assisted generation in one workflow.

7. Proposed System

The proposed system contains three main layers. The Streamlit frontend collects the student's name, subject, topic and learning preferences. The FastAPI backend receives the request, validates it and calls the generation service. The AI generation layer constructs a controlled educational prompt and returns structured content.

The architecture keeps the interface independent from the generation logic. This makes the project easier to test and maintain.

8. Key Features

• Student-friendly topic input
• AI-generated notes
• Key concepts and definitions
• Simple examples
• Revision summary
• Basic study-plan generation
• FastAPI REST API
• Streamlit web interface
• Input validation
• Error handling
• Modular code structure
• Optional export to text or document formats

9. User Journey

1. The student opens the EduAssist application.
2. The student enters a name.
3. The student selects or enters a subject.
4. The student enters a topic.
5. The student chooses a learning level or preferred detail.
6. The application sends the request to FastAPI.
7. The backend validates the request.
8. The AI prompt is constructed.
9. Generated learning material is returned.
10. The student reviews the result and uses it for revision.
11. Optional output can be saved for later use.

10. Technology Stack

Frontend: Streamlit.
Backend: FastAPI with Uvicorn.
Programming Language: Python.
Validation: Pydantic.
AI Layer: Generative AI API through a configurable service function.
HTTP Client: Requests.
Document support: python-docx and reportlab/FPDF can be added for export.
Environment configuration: python-dotenv.

This stack is lightweight and suitable for demonstrating a full AI-enabled application workflow.

11. System Architecture

The architecture follows this sequence:

Student → Streamlit UI → FastAPI /generate endpoint → Request validation → Prompt Builder → AI Generator → Structured learning response → Streamlit preview.

The frontend is responsible for user interaction. The backend is responsible for request processing. The AI service is responsible for content generation. Keeping these roles separate makes the application easier to debug and upgrade.

Architecture/flow representation: Student → Streamlit → FastAPI → AI Generator → Structured Output → Student.

12. Module Description

1. User Interface Module – collects student inputs and displays results.
2. API Module – exposes REST endpoints.
3. Validation Module – checks required fields and input lengths.
4. Prompt Builder – converts structured input into a controlled prompt.
5. AI Generator – sends the prompt to the configured AI provider.
6. Study Planner – organizes topics into a simple schedule.
7. Output Module – formats generated content for display or download.
8. Error Handler – provides readable messages when a request fails.

13. Functional Requirements

FR1: The system shall accept student name, subject and topic.
FR2: The system shall validate required fields.
FR3: The system shall provide a generate-notes operation.
FR4: The system shall return structured learning content.
FR5: The system shall display the generated content.
FR6: The system shall provide a study-plan operation.
FR7: The system shall report errors without crashing the interface.

14. Non-Functional Requirements

Performance: The interface should remain responsive while requests are processed.
Usability: Input labels and outputs should be easy to understand.
Maintainability: Modules should be separated into logical files.
Reliability: Invalid requests should be handled safely.
Security: API keys should be stored in environment variables rather than hard-coded.
Scalability: The backend should allow additional endpoints and AI providers to be added later.

15. Project Workflow

The workflow begins when the student submits a topic. The frontend packages the information as JSON. FastAPI receives the request and validates it with Pydantic. The backend builds a prompt containing the subject, topic and desired output structure. The AI service generates the response. The backend returns the response as JSON. Streamlit displays the result in a readable format.

Architecture/flow representation: Student → Streamlit → FastAPI → AI Generator → Structured Output → Student.

16. AI Prompt Design

Prompt design is important because the application needs consistent educational output. The prompt should specify the subject, topic, academic level and requested structure. It should instruct the model to use clear language, headings, bullet points and examples.

A controlled prompt can request:
• Introduction
• Key definitions
• Main concepts
• Step-by-step explanation
• Example
• Common mistakes
• Quick revision points
• Final summary

The system should avoid presenting generated content as official academic or professional advice.

17. Backend Development

The FastAPI backend provides a simple REST interface. Pydantic models define the expected input. A POST request is used because the client sends structured data to the server.

The initial implementation can use a local mock generator for testing. After the API is verified, the mock function can be replaced with a real Generative AI integration.

18. Frontend Development

Streamlit provides a simple browser-based interface without requiring a separate JavaScript frontend. Text input fields collect the required information, and a button starts generation. The returned JSON is displayed using headings and text areas.

The interface can later be expanded with dropdowns, progress indicators, download buttons and session history.

19. Complete Source Code

The following code provides a working starter implementation. It uses a local mock generator so the project can be tested without an AI API key. The mock generator can later be replaced by a real Generative AI provider.

19.1 Project Folder Structure

```
EduAssist/
■■■ backend/
■   ■■■ main.py
■   ■■■ models.py
■   ■■■ generator.py
■■■ frontend/
■   ■■■ app.py
■■■ .env
■■■ requirements.txt
```

19.2 backend/models.py

```
from pydantic import BaseModel, Field

class StudyRequest(BaseModel):
    student_name: str = Field(min_length=2, max_length=80)
    subject: str = Field(min_length=2, max_length=80)
    topic: str = Field(min_length=2, max_length=150)
    level: str = Field(default="College", max_length=50)
    detail: str = Field(default="Medium", max_length=30)

class PlanRequest(BaseModel):
    student_name: str
    subject: str
    topics: list[str]
    days: int = Field(ge=1, le=30)
```

19.3 backend/generator.py

```
def generate_notes(data):
    return f"""# {data.topic}

Student: {data.student_name}
Subject: {data.subject}
Level: {data.level}

## Introduction
{data.topic} is an important topic in {data.subject}. This section
provides a simple starting explanation for revision.

## Key Concepts
1. Understand the basic definition.
2. Identify the main concepts and terminology.
3. Connect the concept with a practical example.
4. Review the topic using short revision points.

## Example
Study one simple example related to {data.topic} and explain why
the example demonstrates the main concept.

## Quick Revision
- Remember the main definition.
- Learn the important terms.
- Practice one or two examples.
- Review the summary before an assessment.

## Summary
The topic can be understood more effectively by learning the
definition, main concepts, examples and revision points in order."""

def generate_plan(data):
    topics = data.topics
    lines = [f"# Study Plan for {data.student_name}", ""]
    for i, topic in enumerate(topics):
        day = (i % data.days) + 1
        lines.append(f"Day {day}: {data.subject} - {topic}")
        lines.append("  • Read the concept")
        lines.append("  • Write 5 revision points")
```

```
        lines.append("  • Practice one example")
    return "\n".join(lines)
```

19.4 backend/main.py

```
from fastapi import FastAPI, HTTPException
from models import StudyRequest, PlanRequest
from generator import generate_notes, generate_plan

app = FastAPI(
    title="EduAssist API",
    description="AI-powered student learning and study planner",
    version="1.0.0"
)

@app.get("/")
def home():
    return {"message": "EduAssist API is running"}

@app.post("/generate")
def generate(data: StudyRequest):
    try:
        result = generate_notes(data)
        return {"success": True, "generated_notes": result}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))

@app.post("/study-plan")
def study_plan(data: PlanRequest):
    try:
        result = generate_plan(data)
        return {"success": True, "study_plan": result}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))
```

19.5 frontend/app.py

```
import streamlit as st
import requests

API_URL = "http://127.0.0.1:8000"

st.set_page_config(page_title="EduAssist", page_icon="■")
st.title("EduAssist – AI Student Learning Assistant")
st.write("Generate structured study notes and a simple revision plan.")

name = st.text_input("Student Name")
subject = st.text_input("Subject")
topic = st.text_input("Topic")
level = st.selectbox("Academic Level", ["School", "College", "Beginner"])
detail = st.selectbox("Detail Level", ["Short", "Medium", "Detailed"])

if st.button("Generate Notes"):
    if not name or not subject or not topic:
        st.warning("Please fill in all required fields.")
    else:
        response = requests.post(
            f"{API_URL}/generate",
            json={
                "student_name": name,
                "subject": subject,
                "topic": topic,
                "level": level,
                "detail": detail
            },
            timeout=60
        )

        if response.ok:
            data = response.json()
            st.subheader("Generated Notes")
            st.markdown(data["generated_notes"])
        else:
            st.error("Unable to generate notes.")

st.divider()
st.subheader("Quick Study Plan")

topics_text = st.text_area(
```

```
    "Enter topics, one per line",
    placeholder="Topic 1\nTopic 2\nTopic 3"
)
days = st.number_input("Number of days", min_value=1, max_value=30, value=3)

if st.button("Create Study Plan"):
    topics = [x.strip() for x in topics_text.split("\n") if x.strip()]

    if not name or not subject or not topics:
        st.warning("Enter your name, subject and at least one topic.")
    else:
        response = requests.post(
            f"{API_URL}/study-plan",
            json={
                "student_name": name,
                "subject": subject,
                "topics": topics,
                "days": int(days)
            },
            timeout=60
        )

        if response.ok:
            st.markdown(response.json()["study_plan"])
        else:
            st.error("Unable to create the study plan.")
```

19.6 requirements.txt

```
fastapi
uvicorn
streamlit
requests
pydantic
python-dotenv
python-docx
reportlab
```

19.7 Run Commands

```
# Terminal 1
cd EduAssist/backend
uvicorn main:app --reload

# Terminal 2
cd EduAssist/frontend
streamlit run app.py
```

After starting the backend, the FastAPI service runs locally and the Streamlit application connects to it through the configured API URL.

20. API Testing

The root endpoint can be tested first to confirm that the backend is running. The /generate endpoint can then be tested with valid JSON. The /study-plan endpoint can be tested with several topics and a selected number of days.

Example request:
{
"student_name": "Student",
"subject": "Computer Science",
"topic": "Data Structures",
"level": "College",
"detail": "Medium"
}

Expected result: a JSON object containing success=true and generated_notes.

21. Sample Output

For a topic such as Data Structures, the generated material can contain an introduction, definitions, key concepts, an example, revision points and a summary. The exact content depends on the configured generation service.

A study-plan request containing three topics can distribute those topics across the selected number of study days.

22. Error Handling

The application validates required fields before generation. The frontend checks for empty values and the backend uses Pydantic constraints. Network failures are handled by checking the HTTP response and displaying a user-friendly error.

For production use, additional handling should include request timeouts, API rate limits, structured logging and provider-specific error messages.

23. Security and Privacy

API credentials should never be written directly into source code. They should be stored in environment variables or a secure secret manager. The application should collect only information necessary for the requested learning task.

If user history is stored in a future version, the project should define retention rules and protect stored data. Students should also be informed that AI-generated content can contain errors and should be checked against trusted academic material.

24. Testing Strategy

Unit testing can validate prompt-building and planning functions. API testing can verify request validation and response structures. UI testing can verify that buttons, input fields and error messages behave as expected.

Example test cases:
TC01: Empty student name → validation warning.
TC02: Valid note request → generated_notes returned.
TC03: Empty topic list → study-plan validation warning.
TC04: Backend unavailable → frontend error message.
TC05: Multiple topics → each topic appears in the plan.

25. Deployment

For local demonstration, run FastAPI with Uvicorn and Streamlit separately. For a hosted deployment, the backend and frontend can be deployed as separate services or packaged appropriately. Environment variables should be configured through the deployment platform.

Before deployment, update the frontend API URL, configure allowed origins where necessary, protect secrets, add logging and test the production build.

26. Benefits

EduAssist can reduce repetitive note-preparation work, organize learning material and provide a consistent study structure. Its modular architecture also demonstrates how a modern Python application can connect a web interface, API and AI generation layer.

The project is useful as an academic demonstration because it combines software engineering concepts with an applied AI workflow.

27. Limitations

AI-generated educational content may contain factual or contextual mistakes. The current starter implementation uses a mock generator rather than a live AI provider. The basic study planner does not yet consider examination dates, available hours, learning history or performance.

These limitations can be addressed in future versions through stronger validation, source-grounded generation and personalization.

28. Future Enhancements

Possible extensions include:

- Real Generative AI API integration
- PDF/DOCX export
- Student login and history
- Subject-wise dashboards
- Quiz generation
- Flashcard generation
- Progress tracking
- Reminder notifications
- Calendar integration
- Source-based answers using uploaded study material
- Multilingual learning support
- Teacher/admin dashboards

29. Project Story

A college student named Arun has several subjects to revise before an internal assessment. He starts by entering his name, selecting Computer Science and entering the topic Data Structures. EduAssist creates a structured set of notes with definitions, concepts, an example and quick revision points.

Arun then enters three additional topics into the study planner. The system distributes them across a small number of study days. Instead of starting with a blank page, he receives an organized structure that he can review and compare with his textbook and classroom material.

The story demonstrates the intended role of EduAssist: organizing learning tasks and generating a starting point for revision while leaving academic verification and final learning decisions with the student.

30. Conclusion

EduAssist demonstrates an AI-assisted approach to student learning and study planning. The project combines Streamlit for the user interface, FastAPI for backend services, Pydantic for validation and a modular generation layer for educational content.

The implementation provides a practical foundation that can be expanded with a real Generative AI provider, document export, quizzes, progress tracking and personalized study schedules. The project also demonstrates important software engineering principles including separation of concerns, validation, error handling and secure configuration.

31. References and Learning Resources

The project can be developed further using the official documentation for Python, FastAPI, Streamlit, Pydantic and the selected Generative AI provider. During implementation, always check the current documentation for API syntax, authentication requirements and deployment recommendations.

Appendix A – Sample JSON Data

Example note request:

{"student_name":"Arun","subject":"Computer Science","topic":"Data Structures","level":"College","detail":"Medium"}

Example study-plan request:

{"student_name":"Arun","subject":"Computer Science","topics":["Arrays","Linked Lists","Stacks"],"days":3}

Appendix B – Suggested Presentation Points

1. Problem identified.
2. Proposed AI-based solution.
3. Architecture.
4. Technologies used.
5. FastAPI backend.
6. Streamlit interface.
7. AI generation workflow.
8. Testing.
9. Benefits and limitations.
10. Future enhancements.

Appendix C – Viva Questions and Answers

Q1. What is EduAssist?

A. EduAssist is an AI-assisted learning and study-planning application.

Q2. Why is FastAPI used?

A. FastAPI provides a lightweight REST backend for receiving and processing structured requests.

Q3. Why is Streamlit used?

A. Streamlit provides a simple Python-based web interface.

Q4. What does Pydantic do?

A. It validates structured request data.

Q5. Can the mock generator be replaced?

A. Yes. It can be replaced with a real Generative AI service while keeping the API structure.

Q6. What is the main limitation?

A. Generated content can contain errors, so students should verify important information with trusted academic sources.